

מסמך תכנון מערכת "אתגר יומי"

מגישה: תמימה פוגל
מספר זהות: 213111263
תאריך: 19.03.2026

1. תיאור כללי

מערכת "אתגר יומי" היא מערכת Web במודל Client-Server שמטרתה לעודד התמדה, מוטיבציה אישית ושיתופיות בין משתמשים באמצעות ביצוע אתגרים יומיים.

המערכת מאפשרת למשתמשים:

- ליצור אתגרים אישיים, קבוצתיים או ציבוריים
- להצטרף לאתגרים קיימים
- לסמן השלמה יומית
- לנהל קבוצות משתמשים
- להזמין משתמשים (כולל משתמשים חיצוניים)
- לצפות בסטטיסטיקות מתקדמות ובהיסטוריית פעילות
- לנהל פרופיל אישי כולל תמונת משתמש

בנוסף, המערכת כוללת פיצ'רים מתקדמים כגון:

- יצירת תמונות לאתגרים (Upload / AI)
- מערכת תגובות
- התחברות באמצעות Google
- מערכת הזמנות חכמה עם המרה ממשתמש חיצוני לפנימי

2. מטרות המערכת

2.1 מטרות פונקציונליות

- רישום והתחברות (כולל OAuth)
- יצירת וניהול קבוצות
- הזמנת משתמשים (פנימיים וחיצוניים)
- יצירת אתגרים לפי רמות חשיפה:

- אישי (Personal)
- קבוצתי (Group)
- ציבורי (Public)

- הצטרפות לאתגרים
- סימון השלמה יומית
- צפייה בסטטיסטיקות

- כתיבת תגובות
- צפייה בהיסטוריה אישית

2.2 מטרות לא-פונקציונליות

- אבטחה באמצעות JWT והרשאות לפי גישה
- מניעת כפילויות (הצטרפות / השלמה)
- תמיכה ב־Timezone לכל משתמש
- ביצועים גבוהים (React + REST)
- ארכיטקטורה מודולרית (Layered Architecture)

3. טכנולוגיות בשימוש

- Backend: Spring Boot
- Database: PostgreSQL
- ORM: Hibernate + Spring Data JPA
- Security: Spring Security + JWT + OAuth2
- (Frontend: React (SPA
- (API: REST (JSON over HTTP
- (Image Handling: Upload + AI generation (HuggingFace

4. זרימת עבודה במערכת

1. הלקוח שולח בקשת HTTP
2. ה־Controller מקבל את הבקשה
3. קריאה ל־Service
4. ה־Service משתמש ב־Repository
5. נתונים מוחזרים כ־DTO
6. תגובת JSON נשלחת ללקוח
7. שכבת Security מאמתת הרשאות

5. מבנה המערכת

המערכת "אתגר יומי" בנויה בארכיטקטורה שכבתית (Layered Architecture), המבוססת על הפרדה ברורה בין שכבת התצוגה, שכבת ה-API, שכבת הלוגיקה העסקית, שכבת הגישה לנתונים ושכבת האבטחה. גם במסמך התכנון הראשוני הוגדרה ארכיטקטורה שכבתית המבוססת על Spring Boot, Controllers, Services, Repositories, Entities, DTOs, Security, אך במימוש הסופי נוספו שכבות אחרות ושירותים נוספים, וההפרדה ביניהן נעשתה רחבה ומפורטת יותר.

מטרת המבנה השכבתי היא לאפשר:

- תחזוקה נוחה של הקוד
- הרחבה עתידית של המערכת
- בידוד בין לוגיקה עסקית לבין תשתיות
- אכיפת הרשאות וכללים עסקיים במקום מרכזי
- שימוש חוזר בקוד ושיפור הקריאות

5.1 עקרון הארכיטקטורה הכללי

במערכת מתקיים זרם עבודה קבוע:

1. הלקוח (React ב-Frontend) שולח בקשת HTTP לשרת.
2. הבקשה מגיעה ל-Controller המתאים.
3. שכבת האבטחה בודקת אימות משתמש והרשאות בסיסיות.
4. ה-Controller מבצע ולידציה בסיסית ומעביר את הטיפול ל-Service המתאים.
5. ה-Service מפעיל את הלוגיקה העסקית, מבצע בדיקות הרשאה מפורטות, ומשתמש ב-Repositories.
6. ה-Repositories ניגשים למסד הנתונים דרך JPA/Hibernate.
7. התוצאות חוזרות לשכבת ה-Service, ממופות ל-DTOs, ונשלחות דרך ה-Controller כ-JSON ללקוח.

5.2 שכבות המערכת

5.2.1 שכבת ה-Frontend

שכבת ה-Frontend ממומשת כאפליקציית SPA ב-React, כפי שכבר הוגדר בתכנון הראשוני. תפקידה של שכבה זו הוא להציג את מסכי המערכת, לנהל ניווט, לשלוח בקשות לשרת, ולהציג את הנתונים למשתמש.

תחומי האחריות המרכזיים:

- ניהול מסכים וניווט בין עמודים
- שמירת טוקן התחברות
- שליחת בקשות API דרך Axios
- ניהול Server State באמצעות React Query
- טיפול אוטומטי בשגיאות 401 והחזרה למסך התחברות
- הצגת דשבורד, אתגרים, קבוצות, הזמנות, היסטוריה והגדרות

המסכים המרכזיים במימוש:

- Login
- Register
- Forgot Password
- Reset Password
- Dashboard
- Challenges
- Groups
- Invitations
- History
- Settings

הרכיבים המרכזיים מחולקים לפי feature modules, כך שהמבנה שומר על מודולריות ועל הפרדה בין תחומי אחריות.

5.2.2 שכבת ה-Controllers

שכבת ה־Controllers אחראית על חשיפת ה־API של המערכת. במסמך הראשוני הוגדרה שכבה זו כמי שמקבלת בקשות HTTP, מבצעת ולידציה בסיסית ומעבירה את הבקשה לשכבת ה־Service. במימוש הסופי שכבה זו הורחבה וכוללת מספר Controllers ייעודיים לפי תחום פונקציונלי.

עקרונות העבודה של Controllers:

- קבלת בקשות HTTP מהלקוח
- מיפוי HTTP Method + URL לפעולה מתאימה
- קבלת DTOs מהלקוח
- ולידציה בסיסית של קלט
- שליפת המשתמש המחובר כאשר נדרש
- העברת הבקשה ל־Service המתאים
- החזרת תגובת JSON תקנית

מחלקות ה־Controllers במערכת

AuthApiController - אחראי על תהליכי הזדהות ואימות

UserApiController - אחראי על פעולות הקשורות למשתמש המחובר

ChallengeApiController - אחראי על כל פעולות האתגרים

GroupApiController - אחראי על ניהול קבוצות

InviteApiController - אחראי על טיפול בהזמנות שקיבל המשתמש

DashboardApiController - אחראי על שליפת נתוני הדשבורד

HistoryApiController - אחראי על שליפת היסטוריית פעילות של המשתמש לפי טווח תאריכים.

5.3 שכבת ה־Services

זוהי השכבה המרכזית והחשובה ביותר במערכת. במסמך הראשוני תואר כי שכבה זו אחראית על כל הלוגיקה העסקית, כללי ההרשאות, הוולידציות וה־Use Cases. במימוש בפועל שכבה זו הורחבה באופן משמעותי, וכל תחום עסקי קיבל Service ייעודי.

עקרונות העבודה של שכבת ה־Services:

- מימוש הכללים העסקיים של המערכת
- אכיפת הרשאות וגישה

- בדיקות קלט ברמת הדומיין
- עבודה מול Repositories
- תיאום בין כמה ישויות במקביל
- החזרת DTOs או אובייקטי דומיין לשכבות מעל

מחלקות ה־Services במערכת

AuthService

אחריות:

- רישום משתמש חדש
- התחברות משתמש
- יצירת JWT
- לוגיקת OAuth של Google
- יצירת משתמש חדש במקרה של התחברות Google ראשונה

UserService

אחריות:

- שליפת פרטי המשתמש המחובר
- עדכון פרופיל
- טיפול בתמונת פרופיל
- חיפוש משתמשים פעילים
- soft delete למשתמש

GroupService

אחריות:

- יצירת קבוצות
- שליפת קבוצות המשתמש
- שליפת חברי קבוצה
- בדיקת חברות בקבוצה
- הסרת חברים
- יציאה מקבוצה
- מחיקת קבוצה (soft delete)

InviteService

אחריות:

- יצירת הזמנות למשתמשים רשומים
- יצירת הזמנות חיצוניות למיילים שאינם רשומים
- שליפת הזמנות למשתמש
- אישור ודחיית הזמנות
- preview להזמנה
- המרת הזמנה חיצונית להזמנה פנימית לאחר הרשמה

ChallengeService

אחריות:

- יצירת אתגרים
- שליפת אתגרים לפי פילטרים
- שליפת אתגר בודד
- מחיקת אתגר
- בדיקת נראות וגישה לאתגר
- ניהול תמונת אתגר

ParticipationService

אחריות:

- הצטרפות לאתגרים
- בדיקת סטטוס הצטרפות

CompletionService

אחריות:

- סימון השלמה יומית
- שליפת השלמות ליום מסוים

StatsService

אחריות:

- בניית סטטיסטיקות לאתגר
- חישוב מספר משתתפים
- חישוב מספר משלימים
- בניית רשימת מצליחים
- סיכומים חודשיים

DashboardService

אחריות:

- בניית דשבורד אישי
- בניית דשבורד קבוצתי
- חישוב streak
- חישוב נתוני 30 הימים האחרונים
- חישוב active challenges

CommentService

אחריות:

- שליפת תגובות לאתגר
- הוספת תגובה חדשה

HistoryService

אחריות:

- שליפת היסטוריית הצטרפויות והשלמות בטווח תאריכים
- חישוב סיכומים יומיים

PasswordResetService

אחריות:

- יצירת טוקן לאיפוס סיסמה
- שמירת הטוקן
- שליחת מייל
- אימות טוקן
- סימון טוקן כ-`used`

ChallengeImageService

אחריות:

- שמירה וולידציה של תמונות אתגר
- ניהול נתיבי קבצים
- יצירת placeholder במקרה הצורך

HuggingFacelImageService

אחריות:

- יצירת תמונה לאתגר באמצעות שירות AI חיצוני
- החזרת null במקרה של כשל או היעדר קונפיגורציה

5.4 שכבת ה-Repositories

במסמך הראשוני הוגדרה שכבה זו כמי שאחראית על גישה לבסיס הנתונים באמצעות Spring Data JPA . כך גם במימוש בפועל.

תפקידי השכבה:

- גישה לנתונים דרך JPA/Hibernate
- ביצוע CRUD
- שאילתות מותאמות אישית
- שליפה יעילה של קשרים בין ישויות
- צמצום בעיות ביצועים כמו N+1 באמצעות fetch joins

רשימת ה-Repositories

UserRepository

אחריות:

- שליפת משתמש לפי מייל
- בדיקת קיום משתמש פעיל
- חיפוש משתמשים פעילים לפי שם/מייל

GroupRepository

אחריות:

- שליפת קבוצות
- איתור קבוצות לפי בעלים
- עבודה עם soft delete

GroupMemberRepository

אחריות:

- שליפת חברי קבוצה
- בדיקת חברות בקבוצה
- שליפה עם פרטי המשתמש

GroupInviteRepository

אחריות:

- שליפת הזמנות לפי קבוצה או לפי משתמש
- בדיקת קיום הזמנה Pending
- fetch joins למניעת N+1

ExternalGroupInviteRepository

אחריות:

- שמירת הזמנות חיצוניות
- שליפה לפי אימייל/קבוצה/סטטוס
- המרה לאחר רישום

ChallengeRepository

אחריות:

- שליפת אתגרים לפי תאריך
- שליפת אתגרים לפי visibility
- שליפת אתגרים לפי יוצר או קבוצה
- פילטרים לפי טווח תאריכים

ParticipantRepository

אחריות:

- בדיקת הצטרפות משתמש לאתגר
- שליפת משתתפים באתגר

CompletionRepository

אחריות:

- בדיקת השלמה כפולה
- שליפת השלמות לפי תאריך/טווח
- חישובים סטטיסטיים

ChallengeCommentRepository

אחריות:

- שליפת תגובות עם פרטי כותב
- מיון תגובות לפי זמן יצירה

PasswordResetTokenRepository

אחריות:

- שמירת טוקני איפוס
- שליפה לפי טוקן
- בדיקת תוקף ושימוש

5.5 שכבת ה־Entities

במסמך הראשוני הוצגו הישויות המרכזיות של המערכת כמו User, Group, GroupInvite, Challenge, ChallengeParticipant ו־ChallengeCompletion. במימוש בפועל נוספו ישויות חדשות ושדות נוספים.

הישויות המרכזיות

User - מייצגת משתמש רשום במערכת.
כוללת נתוני התחברות, שם, אזור זמן, תמונת פרופיל ומחיקה לוגית.

Group - מייצגת קבוצה בבעלות משתמש.
כוללת שם, תיאור, בעלים, ותמיכה ב־soft delete.

GroupMember - מייצגת קשר בין משתמש לקבוצה.

GroupInvite - מייצגת הזמנה פנימית למשתמש רשום.

ExternalGroupInvite - מייצגת הזמנה לפי מייל למשתמש שטרם נרשם.

Challenge - מייצגת אתגר יומי.

כוללת כותרת, תיאור, visibility, תאריך האתגר, יוצר, קבוצה אופציונלית ותמונה.

ChallengeParticipant - מייצגת הצטרפות משתמש לאתגר.

ChallengeCompletion - מייצגת השלמה של אתגר על ידי משתמש בתאריך מסוים.

ChallengeComment - מייצגת תגובה של משתמש על אתגר.

PasswordResetToken - מייצגת טוקן לאיפוס סיסמה.

5.6 שכבת ה־DTOs

גם במסמך הראשוני הוגדרה שכבת DTOs כשכבה המפרידה בין מודל הנתונים הפנימי לבין מודלי הבקשה/תגובה של ה־API.

תפקידי השכבה:

- למנוע חשיפה ישירה של Entities ללקוח
- לעצב תשובות לפי צורכי ה־Frontend
- להקטין coupling בין בסיס הנתונים לבין ה־API
- לשפר אבטחה וקריאות

סוגי DTOs במערכת:

- DTOs לאימות והרשמה
- DTOs לניהול משתמש
- DTOs לקבוצות והזמנות
- DTOs לאתגרים
- DTOs לסטטיסטיקות
- DTOs לדשבורד
- DTOs לתגובות
- DTOs להיסטוריה

5.7 שכבת האבטחה (Security)

במסמך הראשוני צוין כי שכבת האבטחה אחראית על אימות והרשאות באמצעות JWT ו-Spring Security. במימוש בפועל שכבה זו הורחבה ותומכת גם ב-OAuth.

אחריות השכבה:

- אימות משתמש באמצעות JWT
- בניית SecurityContext
- הגנה על endpoints פרטיים
- מתן גישה חופשית רק ל-OAuth, Swagger, auth, uploads ציבוריים
- שילוב התחברות עם Google

מרכיבים עיקריים:

- JwtAuthenticationFilter
- JwtTokenProvider
- SecurityConfig
- JwtPrincipal
- OAuth2LoginSuccessHandler

חשוב לציין שההרשאות העסקיות העמוקות אינן נשארות רק ברמת Spring Security, אלא נאכפות גם ב-Services, למשל:

- האם המשתמש הוא בעל הקבוצה
- האם המשתמש חבר בקבוצה
- האם למשתמש מותר לצפות באתגר מסוים
- האם מותר לו למחוק תמונה או אתגר

כך מתקבלת אבטחה דו-שכבתית:

1. אבטחת גישה כללית ל-endpoint
2. אבטחת דומיין בתוך הלוגיקה העסקית

5.8 זרימת תלויות בין השכבות

המערכת בנויה כך שהתלויות זורמות בכיוון אחד:

Frontend → Controllers → Services → Repositories → Database

בנוסף:

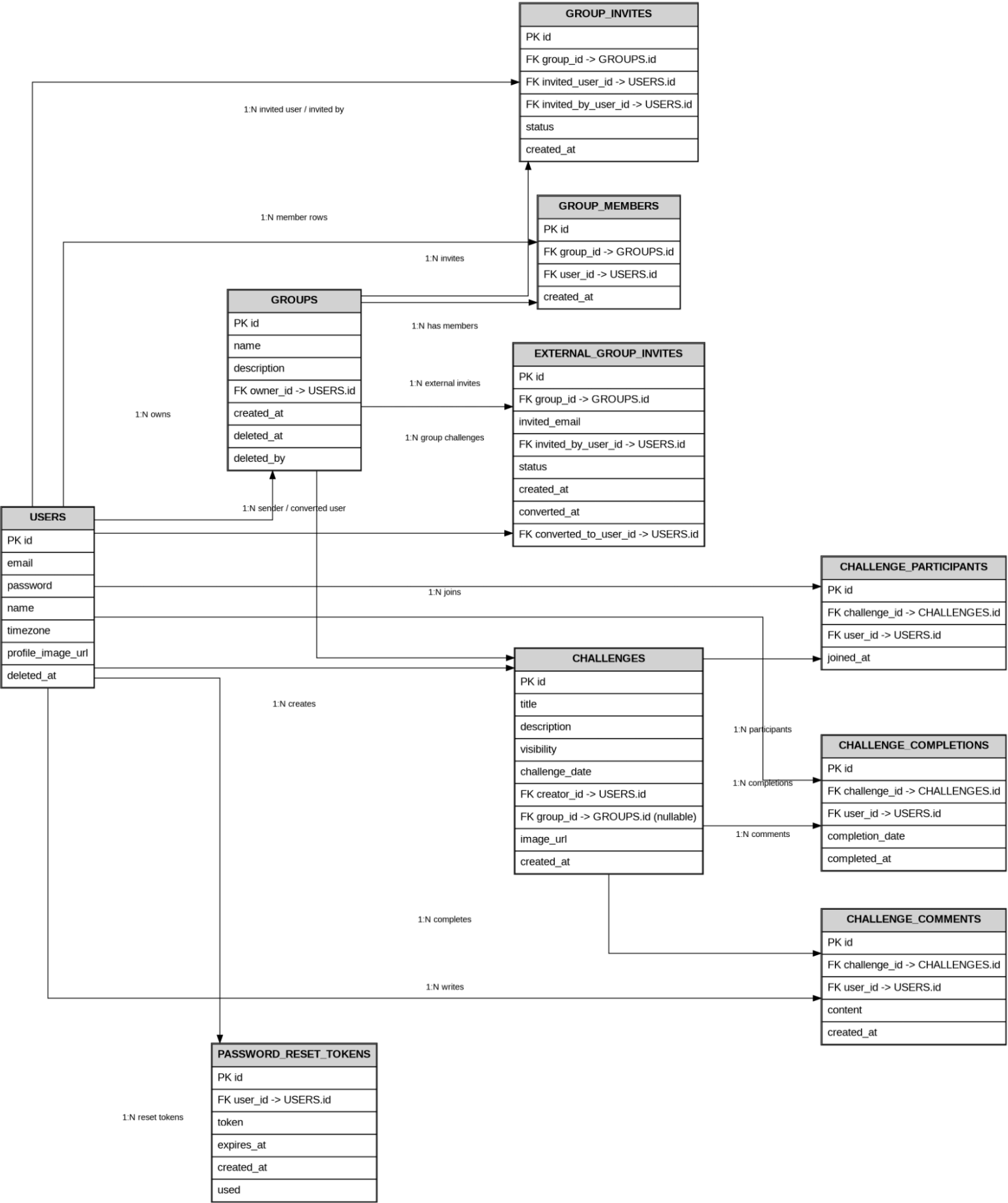
- Controllers תלויים ב־Services בלבד
- Services תלויים ב־Repositories ולעיתים גם ב־Services אחרים
- Repositories אינם תלויים ב־Services
- Entities אינם תלויים ב־Controllers
- DTOs משמשים להעברת מידע בין שכבות
- שכבת Security עוטפת את כל המערכת

6. בסיס הנתונים

במערכת זו ישנו בסיס נתונים השומר בתוכו הן נתונים סטטיים המוכנים מראש והן נתונים דינאמיים אשר נקלטים מהמשתמש. בסיס הנתונים ממומש בעזרת PostgreSQL.

שם הטבלה	מטרת הטבלה	שדות עיקריים	מפתח ראשי / זרים	קשרים מרכזיים
users	שמירת פרטי כל המשתמשים הרשומים במערכת	id, email, password, name, timezone, profileImageUrl, deletedAt	PK: id	משתמש יכול להיות בעלים של קבוצות, יוצר של אתגרים, משתתף ומסיים אתגרים, כותב תגובות, שולח ומקבל הזמנות, ומחזיק טוקנים לאיפוס סיסמה
groups	שמירת פרטי הקבוצות במערכת	id, name, description, ownerId, createdAt, deletedAt, deletedBy	PK: id, FK: ownerId -> users.id	לכל קבוצה יש בעלים אחד, חברים רבים, הזמנות רבות ואתגרים רבים
group_members	מימוש חברות של משתמשים בקבוצות	id, groupId, userId, createdAt	PK: id, FK: groupId -> groups.id, FK: userId -> users.id	מממש קשר רבים־לרבים בין משתמשים לקבוצות
group_invites	שמירת הזמנות פנימיות למשתמשים רשומים	id, groupId, invitedUserId, invitedByUserId, status, createdAt	PK: id, FK: groupId -> groups.id, FK: invitedUserId -> users.id, FK: invitedByUserId -> users.id	כל הזמנה שייכת לקבוצה אחת, למוזמן אחד ולשולח אחד
external_group_invites	שמירת הזמנות חיצוניות למיילים של משתמשים לא רשומים	id, groupId, invitedEmail, invitedByUserId, status, createdAt, convertedAt, convertedToUserId	PK: id, FK: groupId -> groups.id, FK: invitedByUserId -> users.id, FK: convertedToUserId -> users.id	מאפשרת להזמין משתמשים חיצוניים ולהמיר את ההזמנה לאחר הרשמה
challenges	שמירת כלל האתגרים במערכת	id, title, description, visibility, challengeDate, creatorId, groupId, imageUrl, createdAt	PK: id, FK: creatorId -> users.id, FK: groupId -> groups.id	כל אתגר נוצר על ידי משתמש אחד, יכול להיות קשור לקבוצה אחת, וכולל משתתפים, השלמות ותגובות
challenge_participants	שמירת הצטרפות של משתמשים לאתגרים	id, challengeId, userId, joinedAt	PK: id, FK: challengeId -> challenges.id, FK: userId -> users.id	מממש קשר רבים־לרבים בין משתמשים לאתגרים
challenge_completions	שמירת השלמות יומיות של אתגרים	id, challengeId, userId, completionDate, completedAt	PK: id, FK: challengeId -> challenges.id, FK: userId -> users.id	כל השלמה שייכת למשתמש אחד ולאתגר אחד
challenge_comments	שמירת תגובות משתמשים על אתגרים	id, challengeId, userId, content, createdAt	PK: id, FK: challengeId -> challenges.id, FK: userId -> users.id	לכל אתגר יכולות להיות תגובות רבות, וכל תגובה נכתבת על ידי משתמש אחד
password_reset_tokens	שמירת טוקנים זמניים לאיפוס סיסמה	id, userId, token, expiresAt, createdAt, used	PK: id, FK: userId -> users.id	משתמש אחד יכול לקבל לאורך זמן מספר טוקנים לאיפוס סיסמה

ERD



7. מסכים מרכזיים:

